CSC 3215 · WEB TECHNOLOGIES · FINAL-TERM QUIZ 2 · 01 SEPTEMBER

PHP Quiz Revision

Seven blocks, ranked by marks per minute. Syllabus: PHP basics, arrays, loops, functions, classes, forms, validation, superglobals, file upload, cookies, sessions, authentication, AJAX, JSON.

HOW TO USE THIS

1. **Go in order.** Blocks 1–4 are the ones most likely to appear as "write the code" questions. If you run out of time, stop wherever you are — you will still have covered the highest-value material.
2. **Do not just read it.** Reading PHP creates a false feeling of understanding that disappears in front of a blank answer script.
3. **Copy each block out by hand.** Then cover the page and write it again from memory. The second attempt is the one that tells you what you actually know.
4. **Block 5 is free marks.** Three comparison tables, pure recall, nothing to understand. Learn them even if you learn nothing else.
5. **Finish with the checklist** on the last page. Every item is something you write on blank paper, not something you review.

1 Cookies

Small, fixed shape, asked almost every time.

```
setcookie("user", "Minhaz", time() + 86400, "/");   // set, lasts 1 day

echo $_COOKIE["user"];                              // read

setcookie("user", "", time() - 3600, "/");          // delete
```

Three things they test: there is no `deletecookie()` — you delete by setting it again with a *past* time. `setcookie()` must run before any HTML output. A cookie you just set is not readable until the *next* page load.

2 Sessions

Four function calls cover nearly the whole topic.

```
<?php
session_start();                    // ALWAYS the first line, before any HTML

$_SESSION["user"] = "Minhaz";       // save

echo $_SESSION["user"];             // read

session_unset();                    // clear the variables
session_destroy();                  // destroy the session (logout)
?>
```

Guarding a page — blocks anyone not logged in

```php
<?php
session_start();

if (!isset($_SESSION["user"])) {
    header("Location: login.php");
    exit();
}
?>
```

Two marks people lose: every page that touches `$_SESSION` needs its own `session_start()`. And always put `exit();` after `header("Location: ...")`, or the rest of the script keeps running.

3 Form handling and validation

The standard "write a form handler" answer.

```php
<?php
if (isset($_POST["submit"])) {

    $name  = trim($_POST["name"]);
    $email = trim($_POST["email"]);

    if ($name == "" || $email == "") {
        echo "All fields are required.";

    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "Please enter a valid email address.";

    } else {
        echo "Welcome " . htmlspecialchars($name);
    }
}
?>

<form method="POST">
    <input type="text" name="name">
    <input type="text" name="email">
    <button type="submit" name="submit">Send</button>
</form>
```

Remember: the `name` attribute is the key you read in `$_POST` — a field with no `name` is never submitted. `htmlspecialchars()` is what stops XSS, and it is worth a mark on its own.

4 File upload

Fixed shape, short, and a very common code question.

```
<form method="POST" enctype="multipart/form-data">
    <input type="file" name="photo">
    <button type="submit" name="upload">Upload</button>
</form>

<?php
if (isset($_POST["upload"])) {

    $name = $_FILES["photo"]["name"];       // original filename
    $tmp  = $_FILES["photo"]["tmp_name"];   // temporary location
    $size = $_FILES["photo"]["size"];       // bytes
    $ext  = strtolower(pathinfo($name, PATHINFO_EXTENSION));

    if (!in_array($ext, array("jpg", "jpeg", "png"))) {
        echo "Only JPG and PNG are allowed.";

    } elseif ($size > 2000000) {
        echo "File must be under 2 MB.";

    } elseif (move_uploaded_file($tmp, "uploads/" . $name)) {
        echo "Uploaded successfully.";

    } else {
        echo "Upload failed.";
    }
}
?>
```

The two marks that decide this question: `enctype="multipart/form-data"` on the form — without it the file never arrives and `$_FILES` is empty. And `move_uploaded_file()` — without it PHP deletes the temp file when the script ends, so the code “runs fine” and uploads nothing.

5 The three tables — free marks

Pure memorisation. Nothing to understand, and near-certain to appear.

Server-side (PHP) vs client-side (JavaScript)

	PHP — server	JavaScript — client
Runs on	The web server	The user's browser
Code visible	No, only output is sent	Yes, View Source shows it
Database access	Yes	No, not directly
Needs reload	Yes, unless called by AJAX	No
Safe for passwords	Yes	No

GET vs POST

	GET	POST
Data goes	In the URL after `?`	In the request body
Visible	Yes	No
Size limit	About 2000 characters	Effectively unlimited
Bookmarkable	Yes	No
Use for	Search, filters, links	Login, registration, uploads

Session vs cookie

	Session	Cookie
Stored on	The server	The user's browser
Capacity	Effectively unlimited	About 4 KB
Lifetime	Until browser closes or timeout	Until the expiry you set
User can edit	No	Yes
Safe for a user ID	Yes	No

6 Output tracing rules

Where marks quietly leak. Learn the five patterns below.

```
echo "5" + 5;      // 10   the + forces both to numbers
echo "5" . 5;      // 55   the . joins them as text

$x = 5;
echo $x++;         // 5    print first, then increase
echo $x;           // 6
echo ++$x;         // 7    increase first, then print

$i = 10;
do {
    echo $i;       // prints 10 once
    $i++;
} while ($i < 5);  // do...while ALWAYS runs at least once

var_dump(5 == "5");    // true    compares value only
var_dump(5 === "5");   // false   compares value AND type

echo 10 % 3;       // 1    modulus, the remainder
echo strlen("Dhaka");            // 5
echo strtoupper(substr("Bangladesh", -4));   // DESH  negative counts from the end
```

7 JSON, AJAX and classes

Bigger topics, lower priority — but each has one small piece worth knowing.

JSON — two functions

```php
<?php
header("Content-Type: application/json");   // before any output

echo json_encode($array);            // PHP array  ->  JSON text

$arr = json_decode($json, true);     // JSON  ->  PHP array
                                     // the "true" is what gives an ARRAY
                                     // without it you get an object ($obj->name)
?>
```

AJAX — loads data without reloading the page

```
fetch("data.php")
    .then(res => res.json())
    .then(data => {
        document.getElementById("list").innerHTML = data.name;
    });
```

AJAX = Asynchronous JavaScript and XML. It updates part of a page without a full reload, so it is faster and sends less data.

Classes and objects

```php
<?php
class Student {

    public $name;                          // property
    private $cgpa;

    public function __construct($name, $cgpa) {   // runs on "new"
        $this->name = $name;
        $this->cgpa = $cgpa;
    }

    public function show() {               // method
        return $this->name;
    }
}

$s = new Student("Minhaz", 3.7);
echo $s->show();                           // Minhaz
?>
```

Three easy slips: the arrow is `->`, never a dot — the dot joins strings. Write `$this->name`, not `$this->$name`. And `__construct` has *two* underscores.

Visibility: `public` = anywhere · `private` = only inside this class · `protected` = this class and any that extend it.

Final checklist — write these on blank paper

Not reading. Writing. If you cannot produce one closed-book, that is your next hour.

- [ ] The cookie trio — set for a day, read with `isset`, delete with `time() - 3600`.

- [ ] A login page and a guarded page using sessions, with `header` and `exit`.
- [ ] A form handler that checks empty fields and validates an email.
- [ ] A file upload handler, including `enctype` and `move_uploaded_file`.
- [ ] All three comparison tables from memory. Four rows each is enough.
- [ ] The tracing rules: `"5" + 5` vs `"5" . 5`, `$x++` vs `++$x`, `do...while`.
- [ ] A class with a constructor and one method, then create an object and call it.

> **In the exam hall:** if a code question stalls you, write the structure anyway — the `<?php`, the `if (isset($_POST[...]))`, the braces, the variable names. Partial marks are real; an empty answer scores nothing. Start with whichever question you recognise fastest.